GITFLOW LAB — 5

Git • GitHub • VS Code • Branches • Merge • Conflict • Conflict Resolution

1. Before You Start

Read one step at a time. First look at the screenshot. Then read the explanation. After that, perform the same action on your own computer.

2. Five Simple Words

Repository: Your project stored in Git.

Branch: A separate line of work.

Commit: A saved change in Git.

Push: Send your local changes to GitHub.

Merge: Bring changes from one branch into another branch.

3. Gitflow Used in This Lab

The practical uses a main branch, a develop branch, and feature branches.

```
main
  └── develop
        ├── feature/login
        └── feature/conflict
```

The screenshots below show the complete work from creating the GitHub repository, setting up Git, creating branches, committing and pushing changes, creating a conflict, resolving it, and checking the result on GitHub.

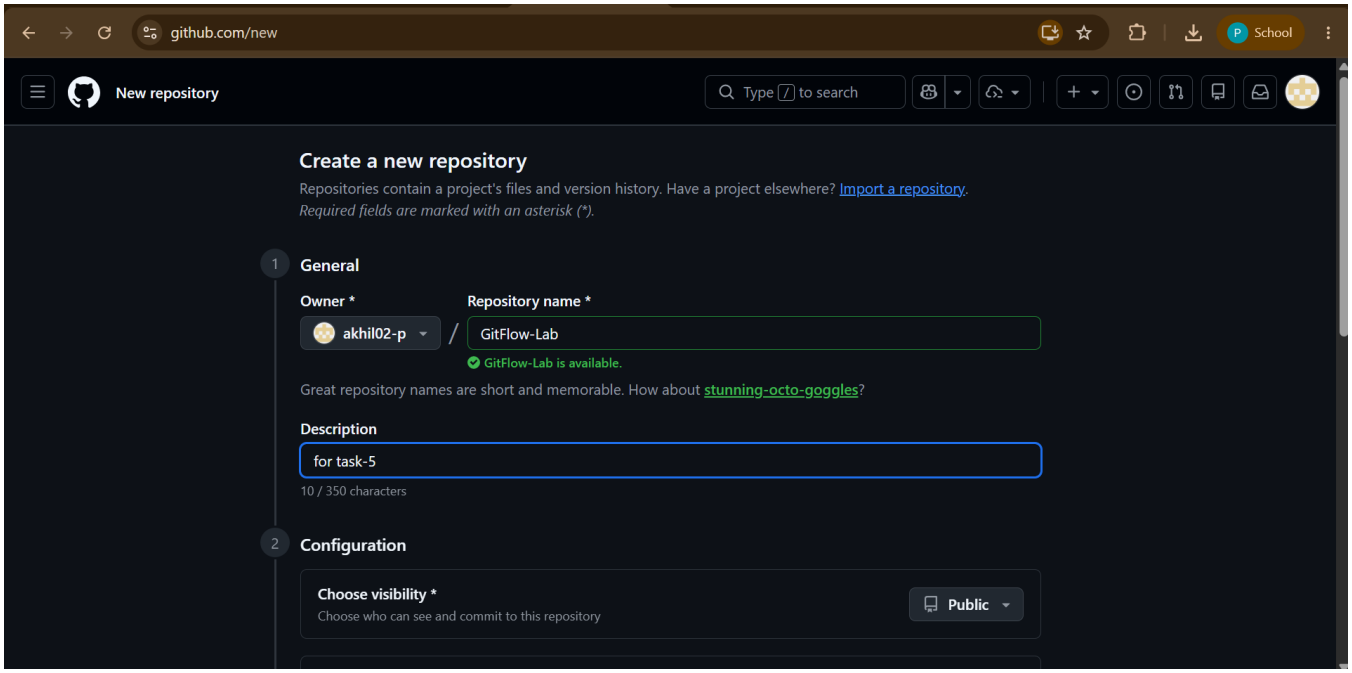
Part A — Screenshot-Based Practical (Steps 1–58)

Each step below shows exactly what appears on screen, a simple explanation of what is happening, the Git command used (where applicable), and the corresponding screenshot from the practical session.

Step 1. Create the GitHub Repository

What you see: Open GitHub and create a new repository named Gitflow-Lab-5. In the screenshot, the repository is Public and the description is "For Task-5".

Simple explanation: This is the first step. The repository is the online place where the lab project will be stored.

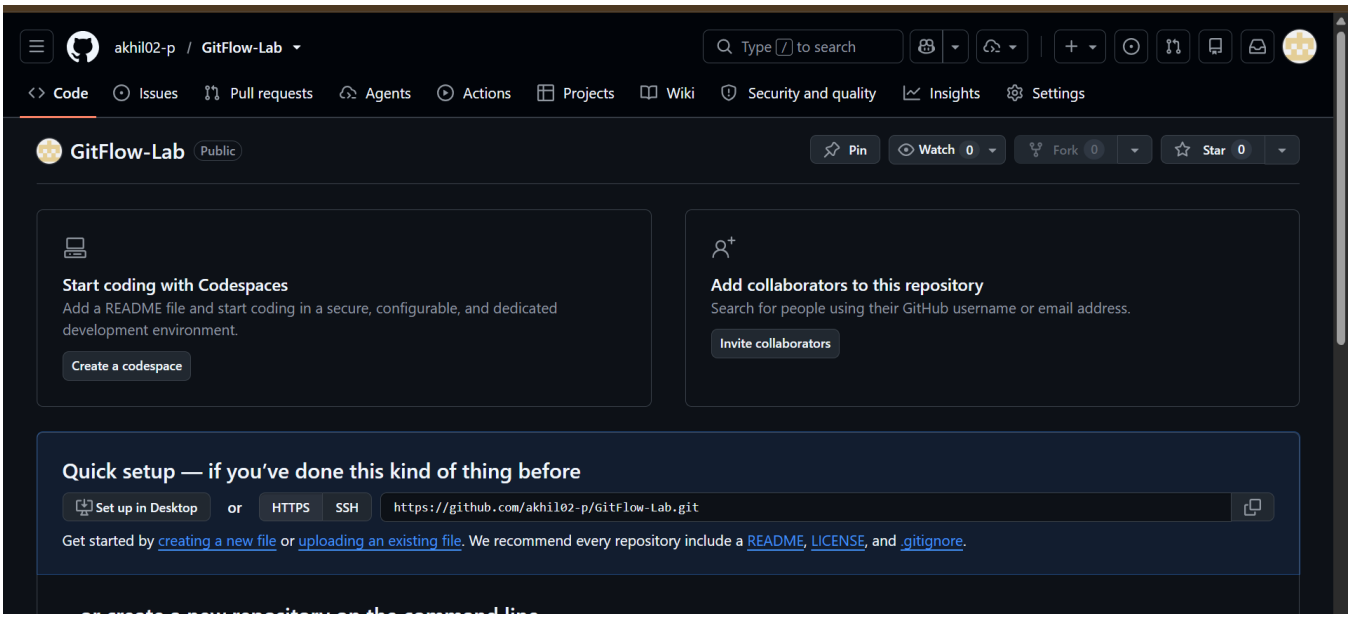


Screenshot 1 — Step 1

Step 2. Open the New Repository

What you see: After creating the repository, GitHub opens the Gitflow-Lab-5 repository page. The HTTPS repository address is visible.

Simple explanation: This page is the home page of the GitHub repository. We will connect the local project to this repository.

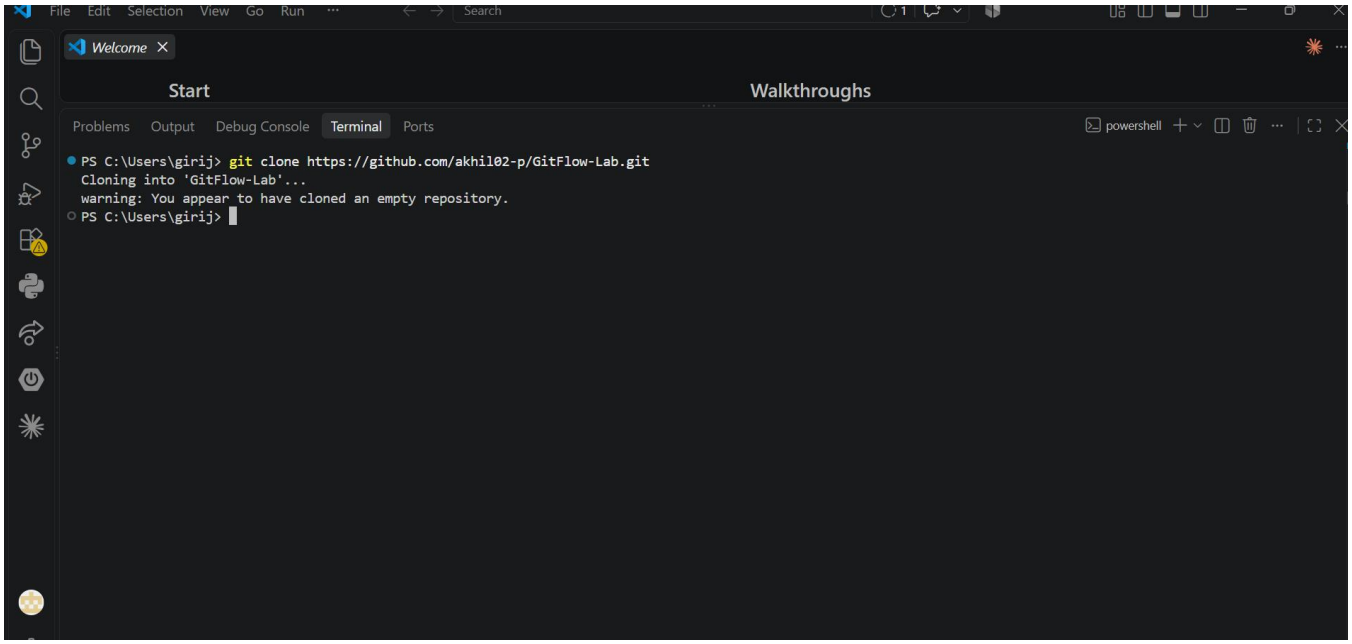


Screenshot 2 — Step 2

Step 3. Understand the Clone Command

What you see: The terminal shows a clone command and a "Repository not found" message for the example address.

Simple explanation: A clone command copies a GitHub repository to the computer. The important idea is that the URL must point to an existing repository.



Screenshot 3 — Step 3

Step 4. Open the Project in VS Code

What you see: The VS Code Explorer shows the FSD workspace and the project folders.

Simple explanation: VS Code is used to work with the files and to run Git commands from the terminal.



Screenshot 4 — Step 4

Step 5. Move to the Project Folder

What you see: The PowerShell terminal moves into D: FSD and then into the Gitflow-Lab-5 folder.

Simple explanation: Git commands should be run inside the project folder.

Command:

```
cd Gitflow-Lab-5
```

Purpose: Moves the terminal into the project folder.



Screenshot 5 — Step 5

Step 6. Create README.md

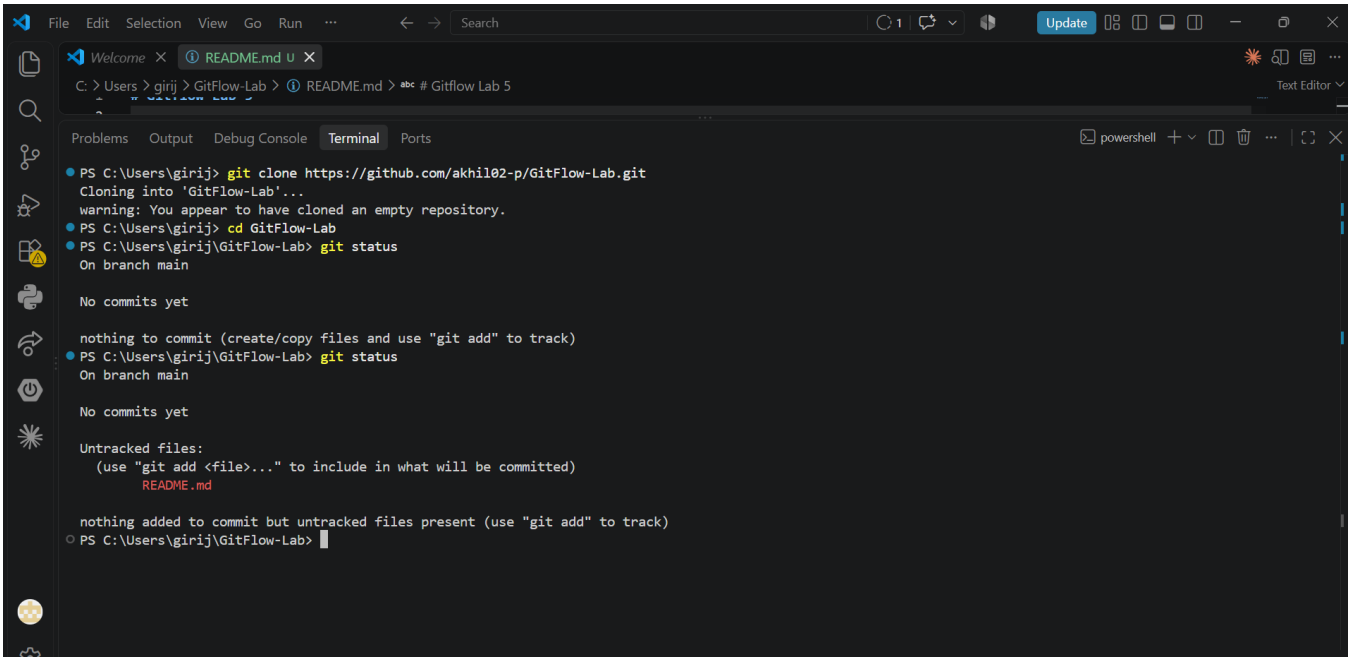
What you see: README.md is created and git status shows it as an untracked file on the main branch.

Simple explanation: README.md is a simple file used to keep information about the project. Git is now detecting it as a new file.

Command:

```
git status
```

Purpose: Checks the current branch and file status.



Screenshot 6 — Step 6

Step 7. Stage README.md

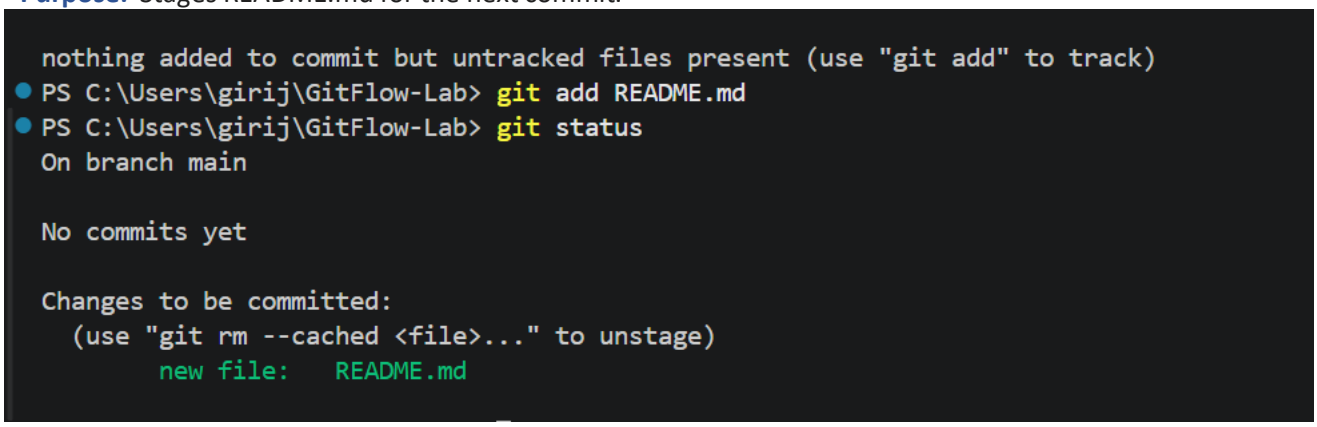
What you see: The command git add README.md is entered.

Simple explanation: git add puts the selected file into the staging area. Staging means the file is ready to be saved in a commit.

Command:

```
git add README.md
```

Purpose: Stages README.md for the next commit.



Screenshot 7 — Step 7

Step 8. Check the Staged File

What you see: git status shows README.md under "Changes to be committed".

Simple explanation: This confirms that README.md has been staged successfully.

Command:

```
git status
```

Purpose: Confirms that README.md is staged.

```
nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\girij\GitFlow-Lab> git add README.md
PS C:\Users\girij\GitFlow-Lab> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
```

Screenshot 8 — Step 8

Step 9. Make the Initial Commit

What you see: The terminal shows the initial commit being created successfully.

Simple explanation: git commit saves the staged changes in Git history. The first saved version is the initial commit.

Command:

```
git commit -m "Initial commit"
```

Purpose: Saves the first project version.

```
        new file:   README.md

PS C:\Users\girij\GitFlow-Lab> git commit -m "Initial commit"
[main (root-commit) 1d20594] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 9 — Step 9

Step 10. Check the Remote Command

What you see: The terminal shows Git's usage/error message after an incorrect remote command.

Simple explanation: A Git command must be written in the correct format. If Git shows usage information, check the command before continuing.

```
 create mode 100644 README.md
PS C:\Users\girij\GitFlow-Lab> git remote -v
origin  https://github.com/akhil02-p/GitFlow-Lab.git (fetch)
origin  https://github.com/akhil02-p/GitFlow-Lab.git (push)
PS C:\Users\girij\GitFlow-Lab> git log --oneline
1d20594 (HEAD -> main) Initial commit
PS C:\Users\girij\GitFlow-Lab> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 225 bytes | 225.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/akhil02-p/GitFlow-Lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\girij\GitFlow-Lab> git checkout -b develop
Switched to a new branch 'develop'
PS C:\Users\girij\GitFlow-Lab> git branch
```

Screenshot 10 — Step 10

Step 11. Check the GitHub Remote

What you see: git remote -v shows the origin repository address.

Simple explanation: origin is the short name used for the GitHub repository connected to the local project.

Command:

```
git remote -v
```

Purpose: Shows the GitHub repository connected as origin.

```
        new file:   README.md

PS C:\Users\girij\GitFlow-Lab> git commit -m "Initial commit"
[main (root-commit) 1d20594] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
PS C:\Users\girij\GitFlow-Lab> git remote -v
origin  https://github.com/akhil02-p/GitFlow-Lab.git (fetch)
origin  https://github.com/akhil02-p/GitFlow-Lab.git (push)
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 11 — Step 11

Step 12. Check the Commit History

What you see: The terminal shows git log --oneline with the Initial commit.

Simple explanation: git log --oneline gives a short view of the saved commits.

Command:

```
git log --oneline
```

Purpose: Shows a short list of commits.

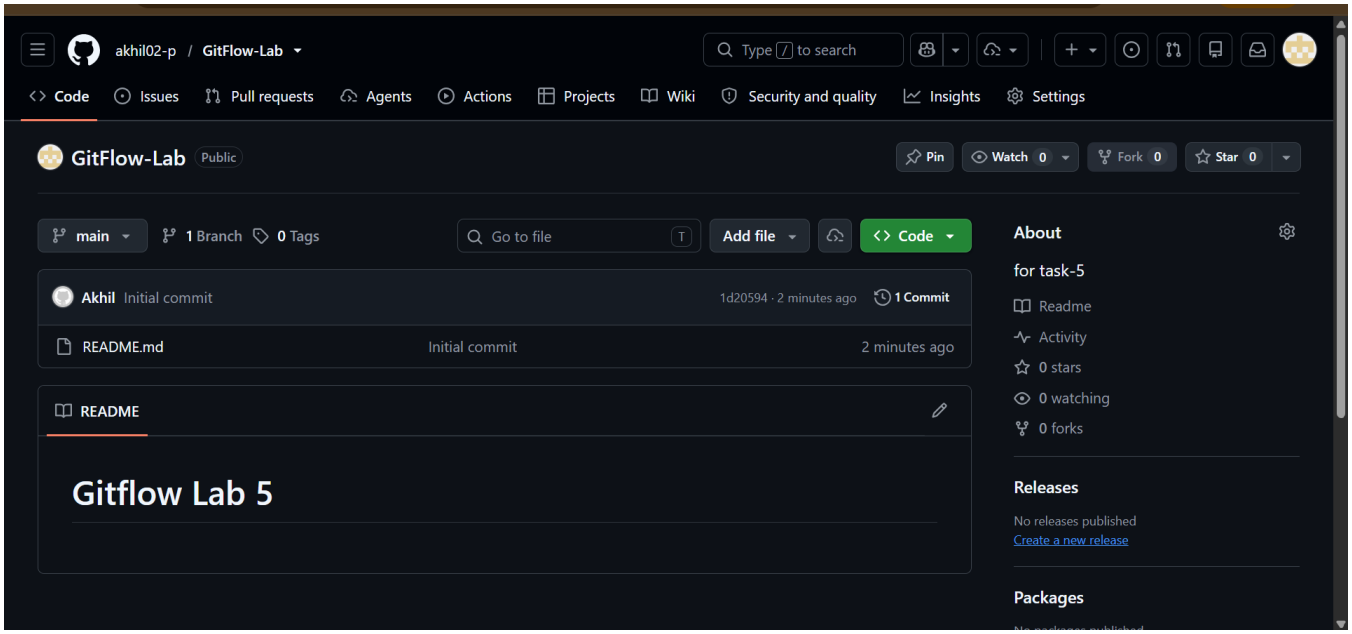
```
PS C:\Users\girij\GitFlow-Lab> git remote -v
origin  https://github.com/akhil02-p/GitFlow-Lab.git (fetch)
origin  https://github.com/akhil02-p/GitFlow-Lab.git (push)
PS C:\Users\girij\GitFlow-Lab> git log --oneline
1d20594 (HEAD -> main) Initial commit
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 12 — Step 12

Step 13. Check the Repository on GitHub

What you see: GitHub shows the main branch, README.md, and the Initial commit.

Simple explanation: This confirms that the repository exists on GitHub and the initial project content is visible.

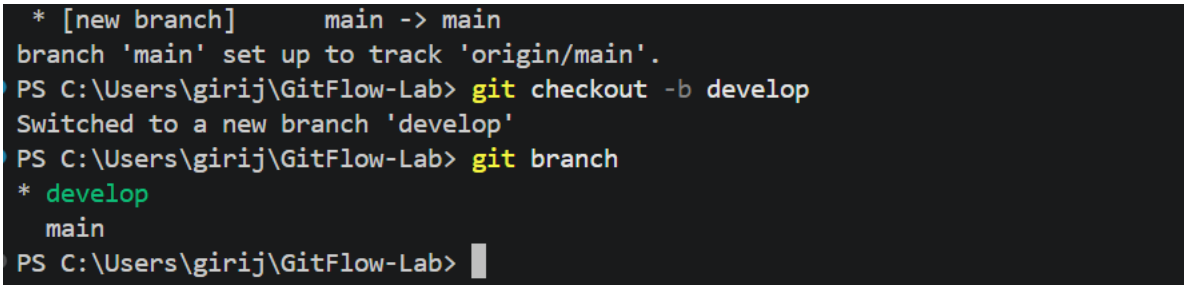


Screenshot 13 — Step 13

Step 14. Check Git Remote Carefully

What you see: The terminal again shows Git's remote command usage/error information.

Simple explanation: If the command is entered incorrectly, Git explains the correct usage. Follow the command format carefully.



Screenshot 14 — Step 14

Step 15. Create the develop Branch

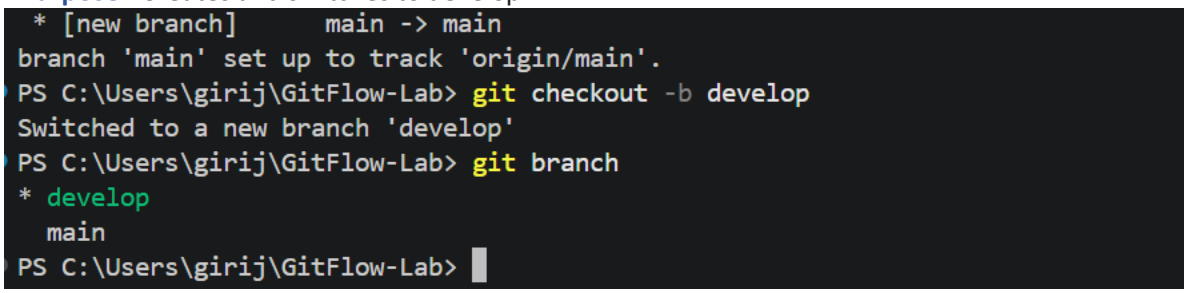
What you see: The command git checkout -b develop creates and switches to the develop branch. git branch lists develop and main.

Simple explanation: develop is the branch used for development work before changes are moved to main.

Command:

```
git checkout -b develop
```

Purpose: Creates and switches to develop.



Screenshot 15 — Step 15

Step 16. Push develop to GitHub

What you see: The terminal shows git push -u origin develop completing successfully.

Simple explanation: The -u option connects the local develop branch with the remote develop branch on GitHub.

Command:

```
git push -u origin develop
```

Purpose: Uploads develop to GitHub and sets its upstream.

```
  main
PS C:\Users\girij\GitFlow-Lab> git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/akhil02-p/GitFlow-Lab/pull/new/develop
remote:
To https://github.com/akhil02-p/GitFlow-Lab.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 16 — Step 16

Step 17. Create feature/login

What you see: The terminal switches from develop to a new feature/login branch.

Simple explanation: A feature branch is used for one particular feature. Here the feature is login.

Command:

```
git checkout -b feature/login
```

Purpose: Creates and switches to feature/login.

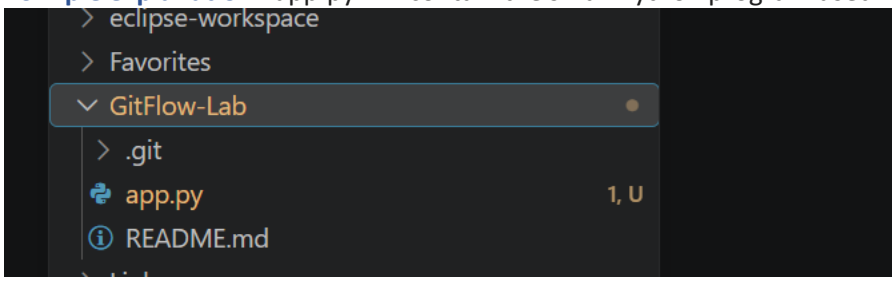
```
  main
PS C:\Users\girij\GitFlow-Lab> git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/akhil02-p/GitFlow-Lab/pull/new/develop
remote:
To https://github.com/akhil02-p/GitFlow-Lab.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
PS C:\Users\girij\GitFlow-Lab> git checkout -b feature/login
Switched to a new branch 'feature/login'
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 17 — Step 17

Step 18. Create app.py

What you see: The VS Code Explorer shows app.py as a new file in Gitflow-Lab-5.

Simple explanation: app.py will contain the small Python program used in this lab.

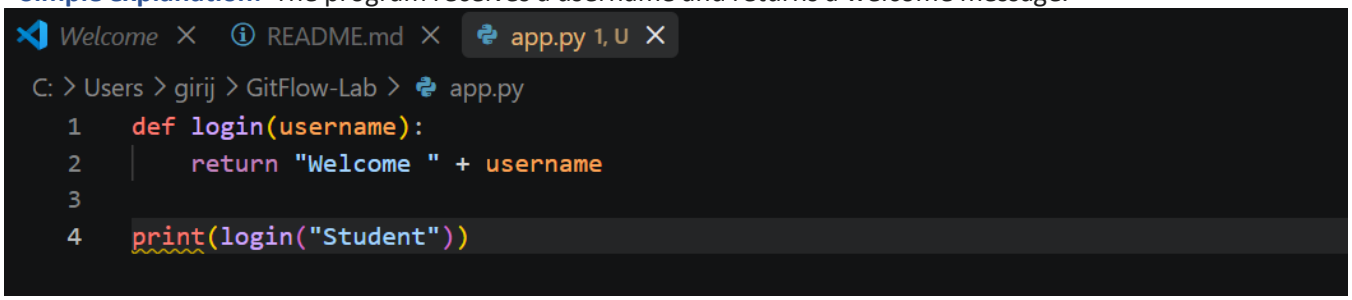


Screenshot 18 — Step 18

Step 19. Write the Login Program

What you see: The editor shows the login function and print(login("Student")) in app.py.

Simple explanation: The program receives a username and returns a welcome message.

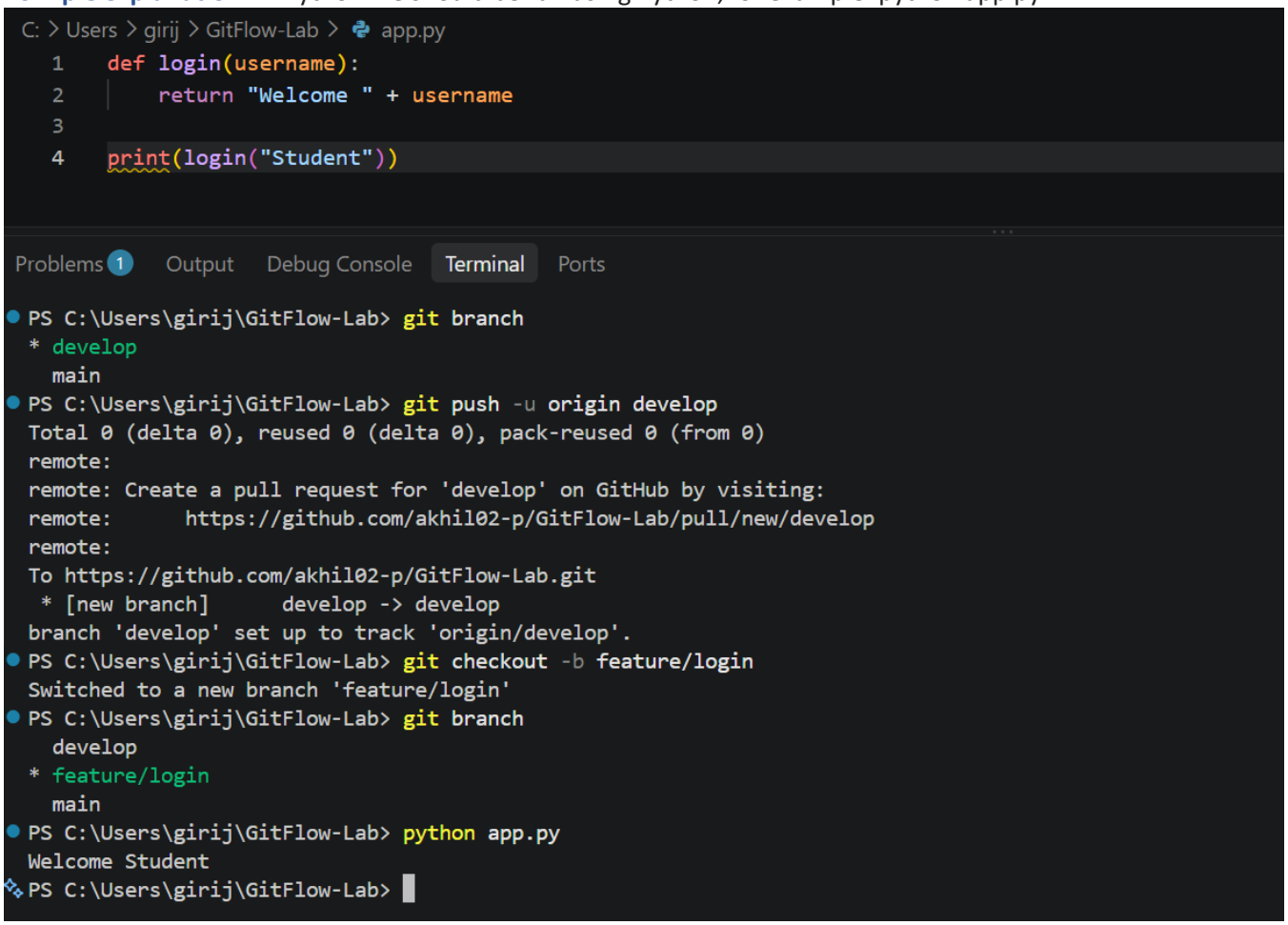


Screenshot 19 — Step 19

Step 20. Check the Python Command

What you see: The terminal shows an error after app.py was entered directly as a PowerShell command.

Simple explanation: A Python file should be run using Python, for example: python app.py.



Screenshot 20 — Step 20

Step 21. Run app.py

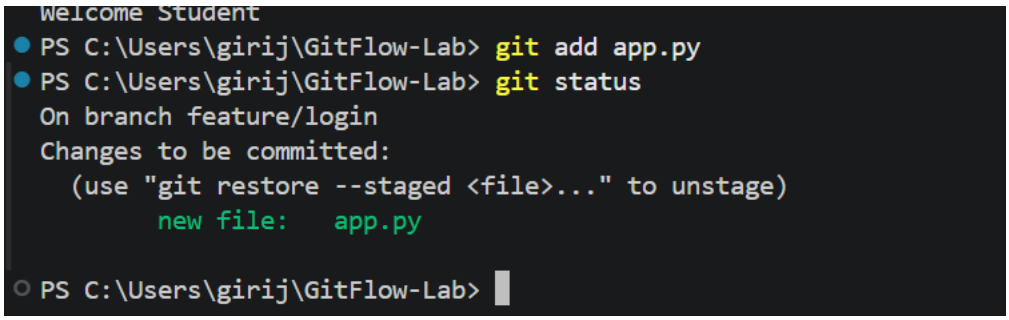
What you see: The terminal shows python app.py producing "Welcome Student", followed by git status showing app.py as untracked.

Simple explanation: The program is working. Git also knows that app.py is a new file that has not yet been staged.

Command:

```
python app.py
```

Purpose: Runs the Python program.

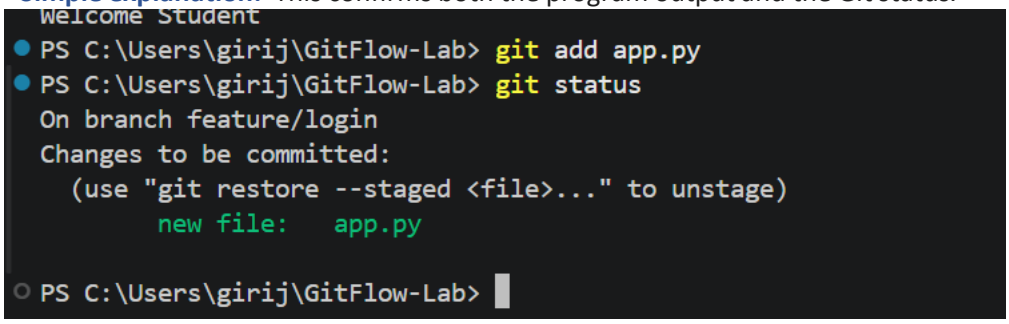


Screenshot 21 — Step 21

Step 22. Check the New File Again

What you see: The terminal again shows python app.py producing "Welcome Student" and git status showing app.py as untracked.

Simple explanation: This confirms both the program output and the Git status.



Screenshot 22 — Step 22

Step 23. Commit the Login Feature

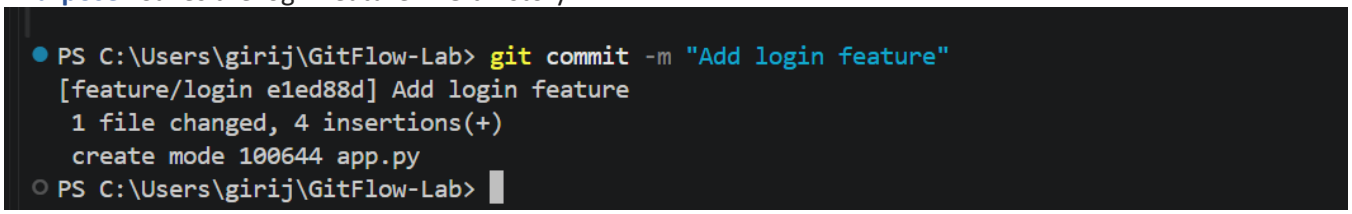
What you see: The command git commit -m "Add login feature" creates the feature/login commit.

Simple explanation: The commit message tells us what was added. This saves the login feature in Git history.

Command:

```
git commit -m "Add login feature"
```

Purpose: Saves the login feature in Git history.



Screenshot 23 — Step 23

Step 24. Stage app.py

What you see: The terminal shows git add app.py and git status with app.py staged.

Simple explanation: The file is now ready to be included in the next commit.

Command:

```
git add app.py
```

Purpose: Stages app.py.

```
PS C:\Users\girij\GitFlow-Lab> git status
PS C:\Users\girij\GitFlow-Lab> git commit -m "Add login feature"
[feature/login e1ed88d] Add login feature
 1 file changed, 4 insertions(+)
 create mode 100644 app.py
PS C:\Users\girij\GitFlow-Lab> git push -u origin feature/login
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 336 bytes | 336.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/login' on GitHub by visiting:
remote:      https://github.com/akhil02-p/GitFlow-Lab/pull/new/feature/login
remote:
To https://github.com/akhil02-p/GitFlow-Lab.git
 * [new branch]      feature/login -> feature/login
branch 'feature/login' set up to track 'origin/feature/login'.
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 24 — Step 24

Step 25. Check and Push feature/login

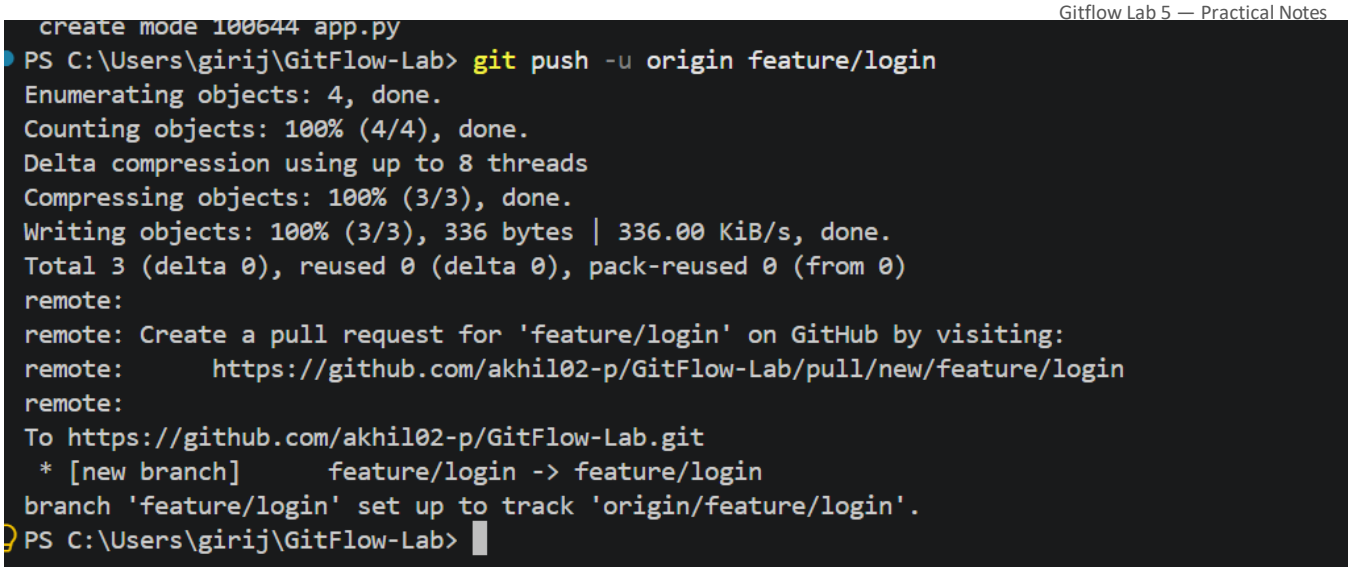
What you see: The terminal shows the Add login feature commit, the short log, and the feature/login push.

Simple explanation: This step checks the feature history and sends the feature branch to GitHub.

Command:

```
git push -u origin feature/login
```

Purpose: Uploads feature/login to GitHub.

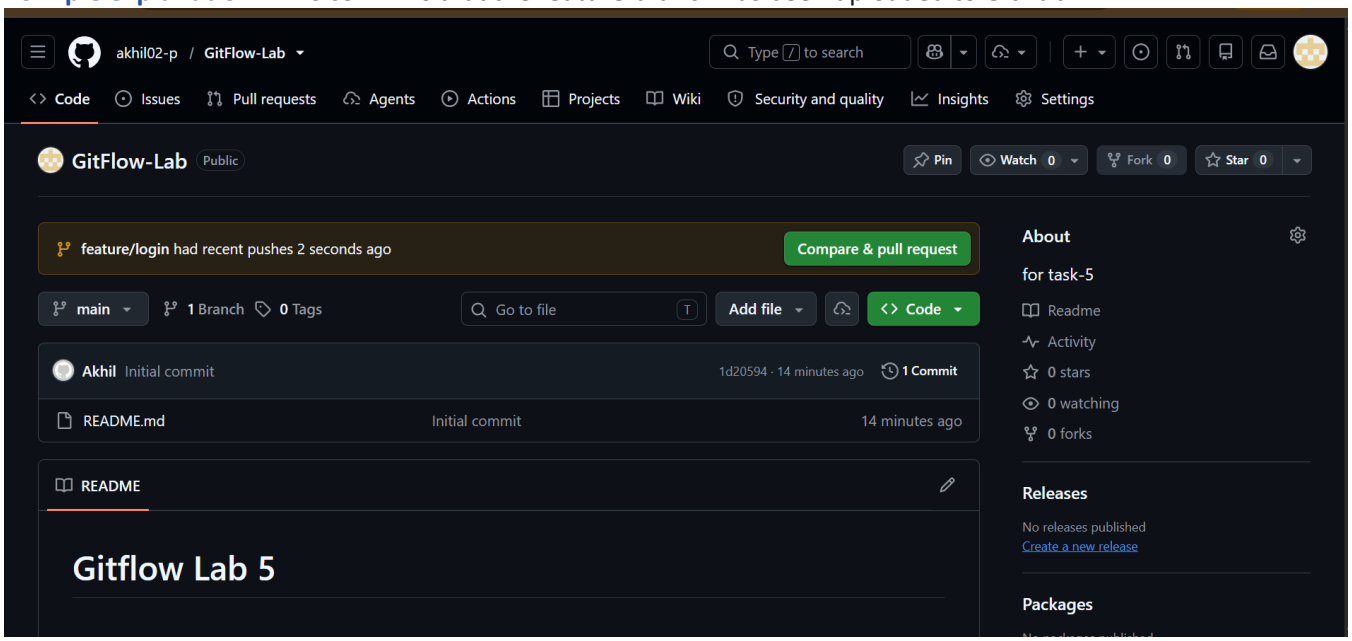


Screenshot 25 — Step 25

Step 26. See feature/login on GitHub

What you see: The GitHub repository page shows a recent push for feature/login.

Simple explanation: This confirms that the feature branch has been uploaded to GitHub.

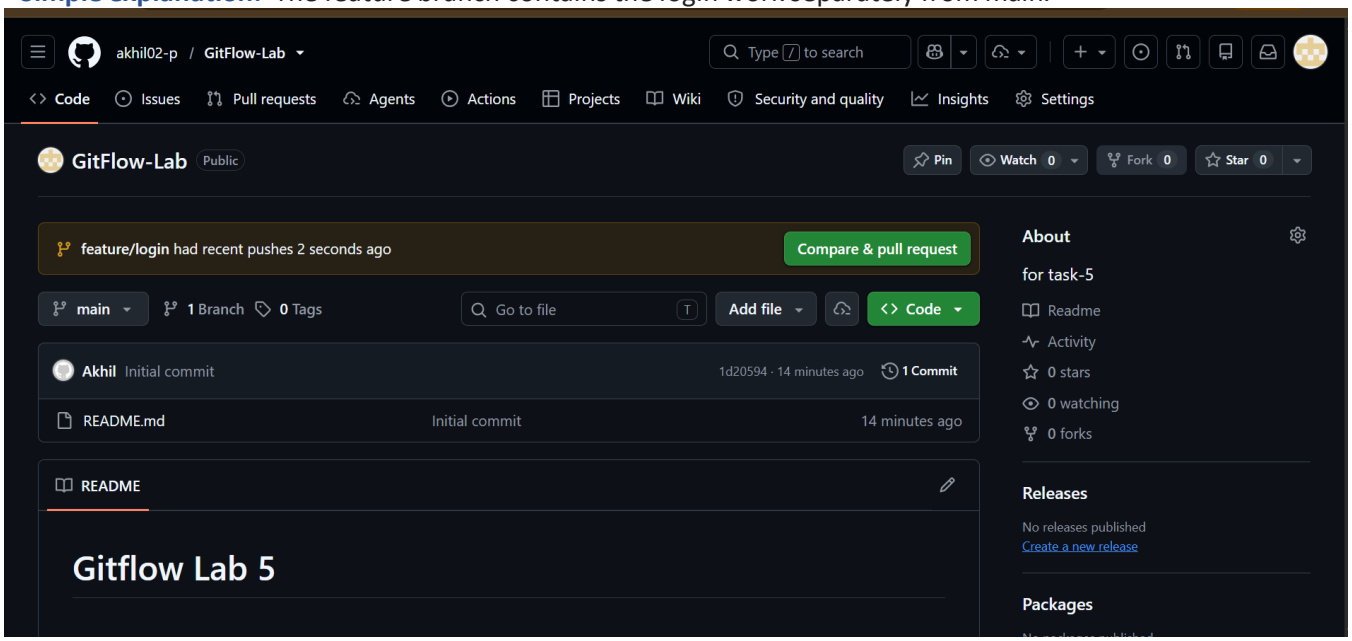


Screenshot 26 — Step 26

Step 27. Open feature/login Branch

What you see: GitHub shows feature/login with app.py and README.md and indicates that the branch is ahead of main.

Simple explanation: The feature branch contains the login work separately from main.



Screenshot 27 — Step 27

Step 28. Review feature/login History

What you see: The terminal shows the staged app.py, the Add login feature commit, and the Git history.

Simple explanation: This is a quick check that the feature work was saved correctly.

Command:

```
git log --oneline
```

Purpose: Shows the short commit history.

```
PS C:\Users\girij\GitFlow-Lab> git status

PS C:\Users\girij\GitFlow-Lab> git commit -m "Resolve README conflict"
[develop 72eb879] Resolve README conflict
PS C:\Users\girij\GitFlow-Lab> git push origin develop
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 809 bytes | 161.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/akhil02-p/GitFlow-Lab.git
   e1ed88d..72eb879  develop -> develop
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 28 — Step 28

Step 29. Finish the feature/login Push

What you see: The terminal shows the push progress and successful upload to GitHub.

Simple explanation: The remote feature/login branch is now updated.

```
PS C:\Users\girij\GitFlow-Lab> git push origin develop
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 809 bytes | 161.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/akhil02-p/GitFlow-Lab.git
   e1ed88d..72eb879  develop -> develop
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
PS C:\Users\girij\GitFlow-Lab> git log --oneline --all
72eb879 (HEAD -> develop, origin/develop) Resolve README conflict
bf1dee0 Update README on develop
9cae6cb (feature/conflict) Update README for conflict feature
e1ed88d (origin/feature/login, feature/login) Add login feature
1d20594 (origin/main, main) Initial commit
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 29 — Step 29

Step 30. Merge feature/login into develop

What you see: The terminal shows git merge feature/login on develop and the message Fast-forward.

Simple explanation: The login feature is now included in develop. Fast-forward means Git could move develop forward without creating a separate merge commit.

Command:

```
git merge feature/login
```

Purpose: Combines feature/login into the current develop branch.

```
PS C:\Users\girij\GitFlow-Lab> git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
PS C:\Users\girij\GitFlow-Lab> git merge feature/login
Updating 1d20594..e1ed88d
Fast-forward
 app.py | 4 ++++
 1 file changed, 4 insertions(+)
 create mode 100644 app.py
PS C:\Users\girij\GitFlow-Lab> git push origin develop
```

Screenshot 30 — Step 30

Step 31. Push Updated develop

What you see: The terminal shows git log --oneline and git push origin develop completing successfully.

Simple explanation: The updated develop branch is now available on GitHub.

Command:

```
git push origin develop
```

Purpose: Uploads the updated develop branch.

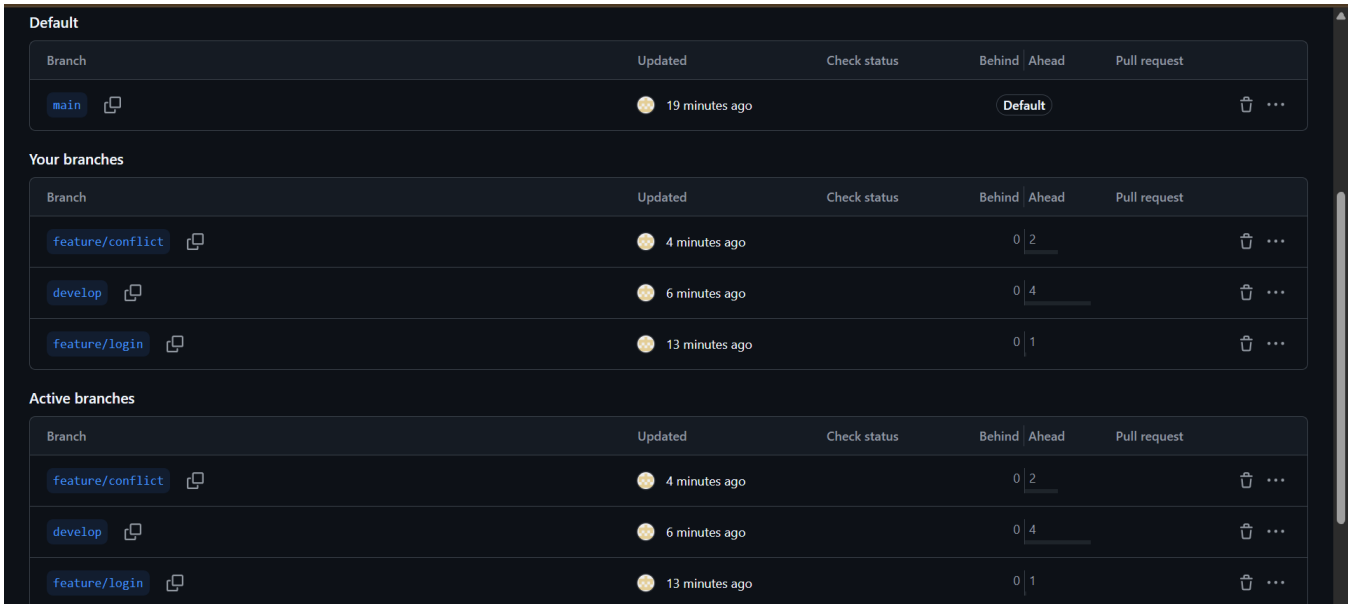
```
feature/login
main
S C:\Users\girij\GitFlow-Lab> # Gitflow Lab 5 - Feature Version
S C:\Users\girij\GitFlow-Lab> git add README.md
S C:\Users\girij\GitFlow-Lab> git status
n branch feature/conflict
hanges to be committed:
 (use "git restore --staged <file>..." to unstage)
       modified:   README.md
```

Screenshot 31 — Step 31

Step 32. Check GitHub Branches

What you see: GitHub shows main, develop, and feature/login in the branch list.

Simple explanation: This is the Gitflow branch structure created so far.



Screenshot 32 — Step 32

Step 33. Prepare the Conflict Branch

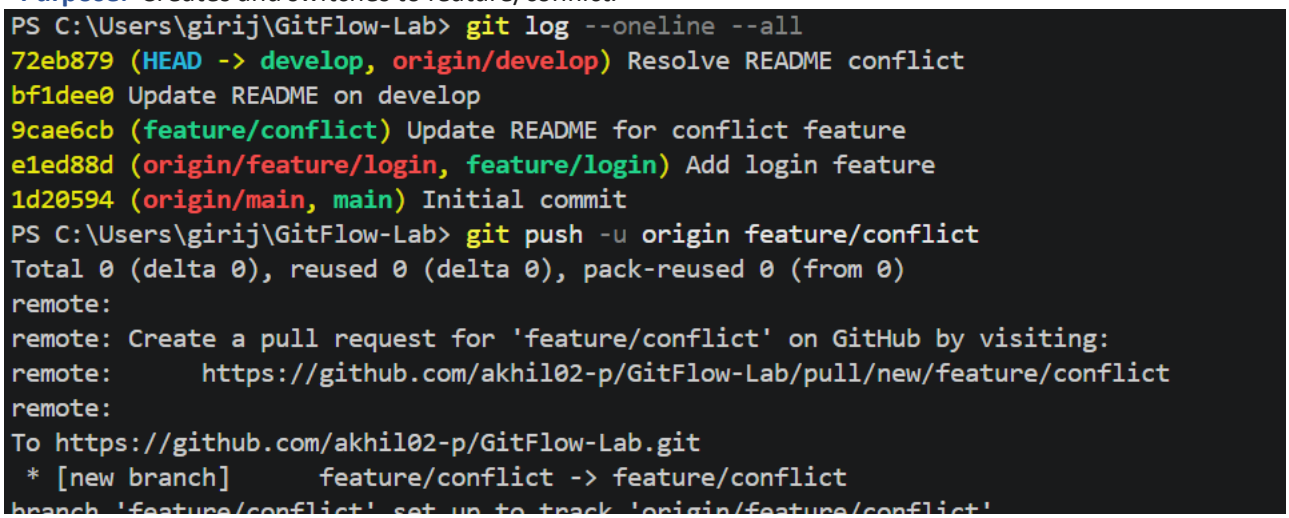
What you see: The terminal shows branch history, a develop push, the branch list, and creation of feature/conflict.

Simple explanation: feature/conflict will be used to demonstrate a merge conflict.

Command:

```
git checkout -b feature/conflict
```

Purpose: Creates and switches to feature/conflict.

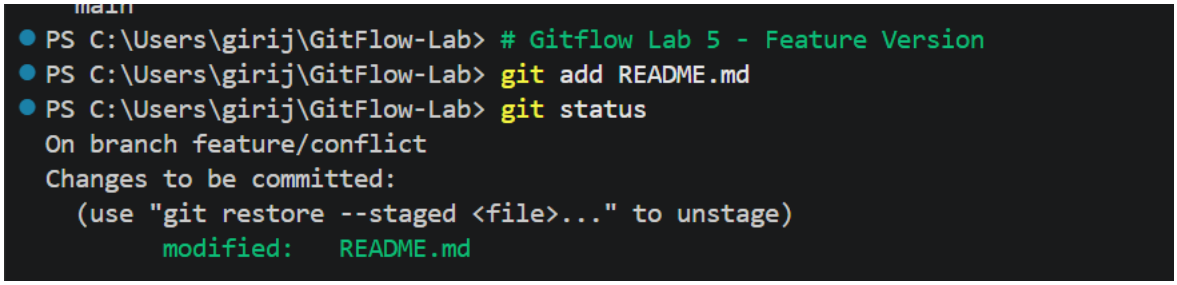


Screenshot 33 — Step 33

Step 34. Open README.md

What you see: VS Code shows README.md with the Gitflow Lab 5 heading while working on the conflict branch.

Simple explanation: README.md will be changed on different branches so Git has two different versions to compare.



Screenshot 34 — Step 34

Step 35. Change the Feature Version

What you see: README.md shows "Gitflow Lab 5 - Feature Version".

Simple explanation: This creates the feature-side version of the README file.

```
main
PS C:\Users\girij\GitFlow-Lab> # Gitflow Lab 5 - Feature Version
PS C:\Users\girij\GitFlow-Lab> git add README.md
PS C:\Users\girij\GitFlow-Lab> git status
On branch feature/conflict
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

Screenshot 35 — Step 35

Step 36. Switch to feature/conflict

What you see: The terminal shows git checkout -b feature/conflict and confirms the branch switch.

Simple explanation: The conflict work is being done on this feature branch.

```
main
PS C:\Users\girij\GitFlow-Lab> # Gitflow Lab 5 - Feature Version
PS C:\Users\girij\GitFlow-Lab> git add README.md
PS C:\Users\girij\GitFlow-Lab> git status
On branch feature/conflict
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

Screenshot 36 — Step 36

Step 37. Stage README.md on feature/conflict

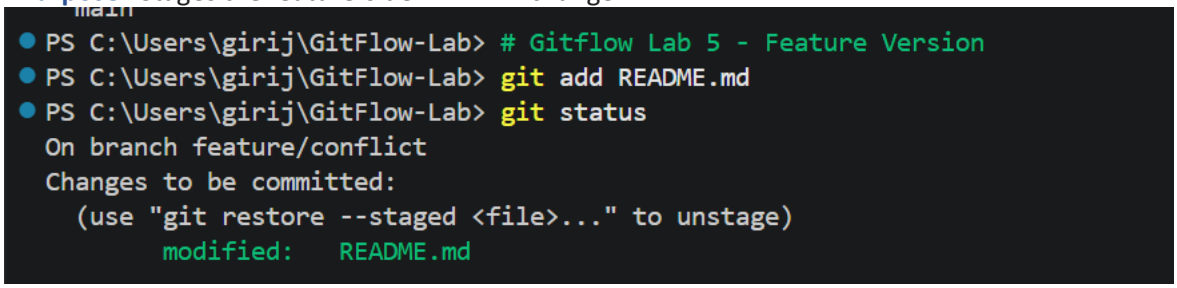
What you see: git add README.md is used and git status shows README.md staged.

Simple explanation: The feature-side README change is ready to commit.

Command:

```
git add README.md
```

Purpose: Stages the feature-side README change.



Screenshot 37 — Step 37

Step 38. Commit the Conflict Feature

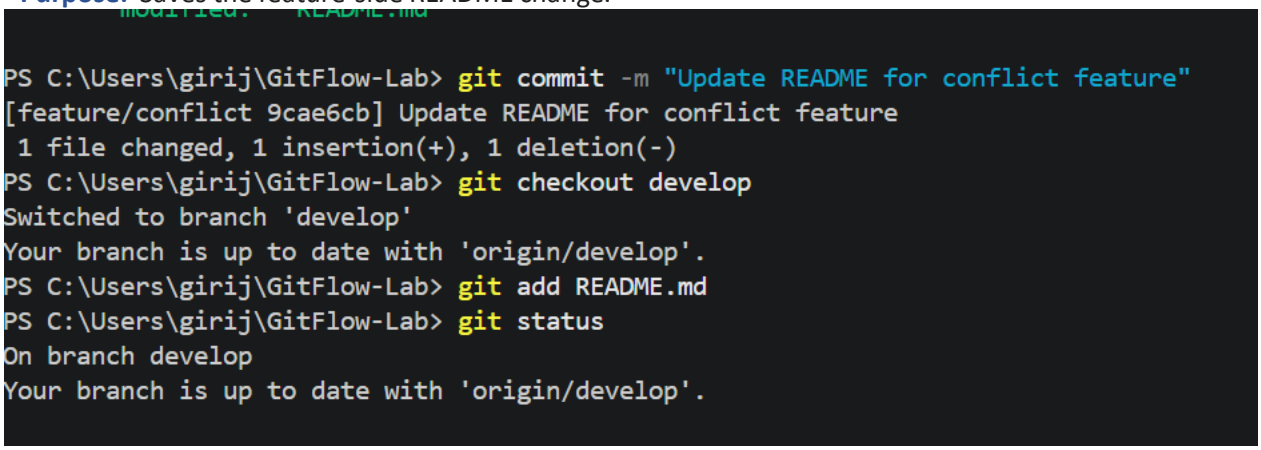
What you see: The terminal shows the commit command for the conflict feature.

Simple explanation: The feature-side README version is saved in Git history.

Command:

```
git commit -m "Update README for conflict feature"
```

Purpose: Saves the feature-side README change.



Screenshot 38 — Step 38

Step 39. Confirm the Conflict Feature Commit

What you see: The terminal shows the successful "Update README for conflict feature" commit.

Simple explanation: The feature/conflict branch now has its own README change.

```
        modified:   README.md

PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README for conflict feature"
[feature/conflict 9cae6cb] Update README for conflict feature
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
PS C:\Users\girij\GitFlow-Lab> git add README.md
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is up to date with 'origin/develop'.
```

Screenshot 39 — Step 39

Step 40. Create the Develop Version

What you see: README.md shows "Gitflow Lab 5 - Develop Version".

Simple explanation: Now a different README version is prepared on develop.

```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Screenshot 40 — Step 40

Step 41. Review the Branch History

What you see: VS Code Source Control/Timeline shows develop, feature/conflict, feature/login, and main in the history.

Simple explanation: The history helps us see that different branches contain different changes.

```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Screenshot 41 — Step 41

Step 42. Review the Project History

What you see: VS Code shows the Gitflow-Lab-5 project and the recorded README changes.

Simple explanation: The Timeline is useful for seeing saved file changes over time.

```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Screenshot 42 — Step 42

Step 43. Continue the Develop Change

What you see: README.md again shows the develop version in VS Code.

Simple explanation: This is the develop-side README content that will be different from the feature version.

```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Screenshot 43 — Step 43

Step 44. Check develop Status

What you see: git status shows README.md modified on develop.

Simple explanation: Git has detected the change and it is ready to be staged.

Command:

```
git status
```

Purpose: Shows that README.md is modified.

```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Screenshot 44 — Step 44

Step 45. Stage the Develop Change

What you see: VS Code Source Control shows the README.md change staged.

Simple explanation: The develop-side change is now ready for commit.

```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   README.md
```

Screenshot 45 — Step 45

Step 46. Commit the Develop README

What you see: The terminal shows git add README.md, git status, and the commit command.

Simple explanation: This saves the develop-side README change in Git history.

Command:

```
git commit -m "Update README on develop"
```

Purpose: Saves the develop-side README change.


```
PS C:\Users\girij\GitFlow-Lab> git commit -m "Update README on develop"
[develop bf1dee0] Update README on develop
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\girij\GitFlow-Lab> git merge feature/conflict
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\girij\GitFlow-Lab> git status
On branch develop
Your branch is ahead of 'origin/develop' by 1 commit.
  (use "git push" to publish your local commits)

All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

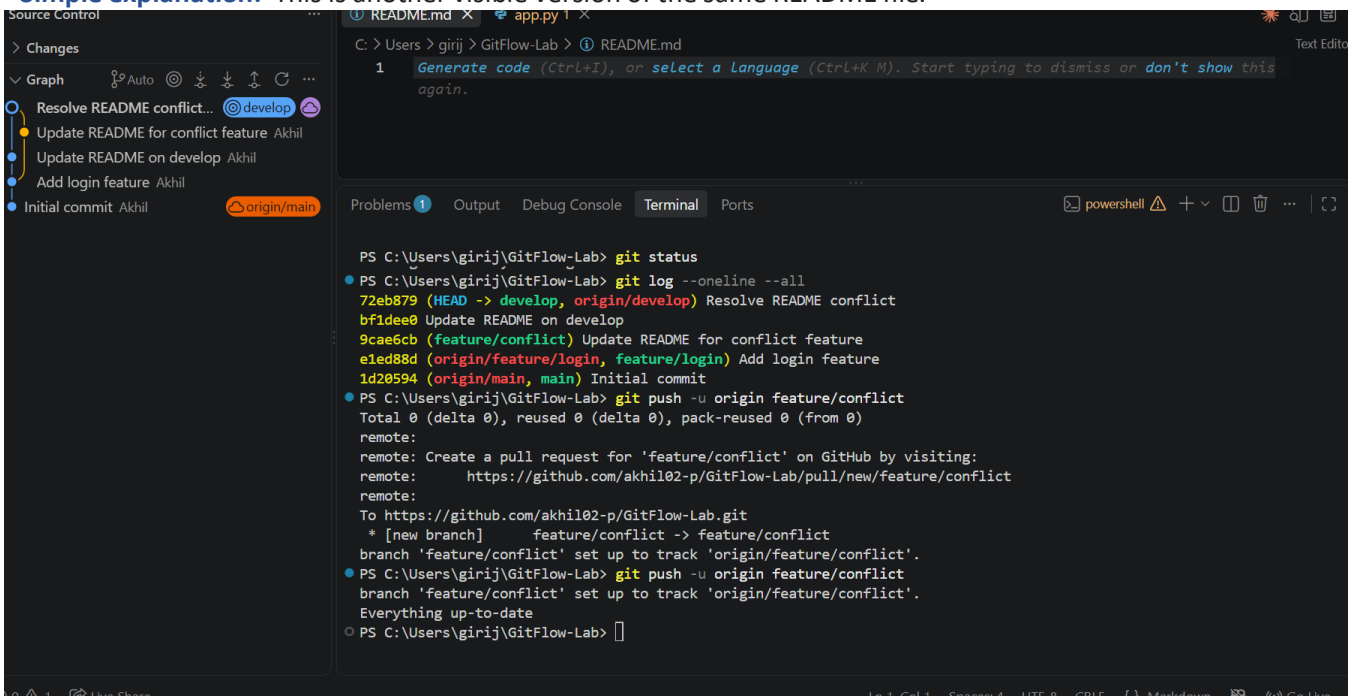
Changes to be committed:
        modified:   README.md
```

Screenshot 46 — Step 46

Step 47. Create the Conflict Version

What you see: README.md shows "Gitflow Lab 5 - Conflict Version".

Simple explanation: This is another visible version of the same README file.

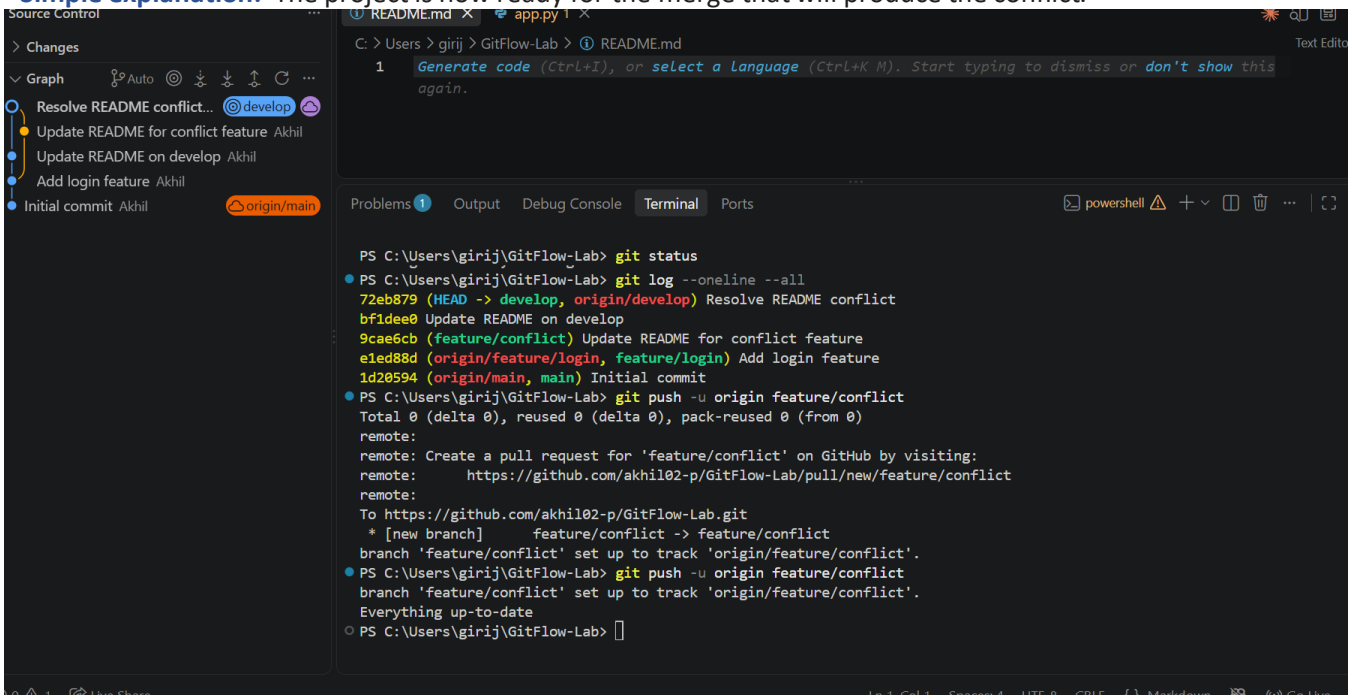


Screenshot 47 — Step 47

Step 48. Open the Merge State

What you see: VS Code shows README.md in the conflict workflow and Source Control information.

Simple explanation: The project is now ready for the merge that will produce the conflict.

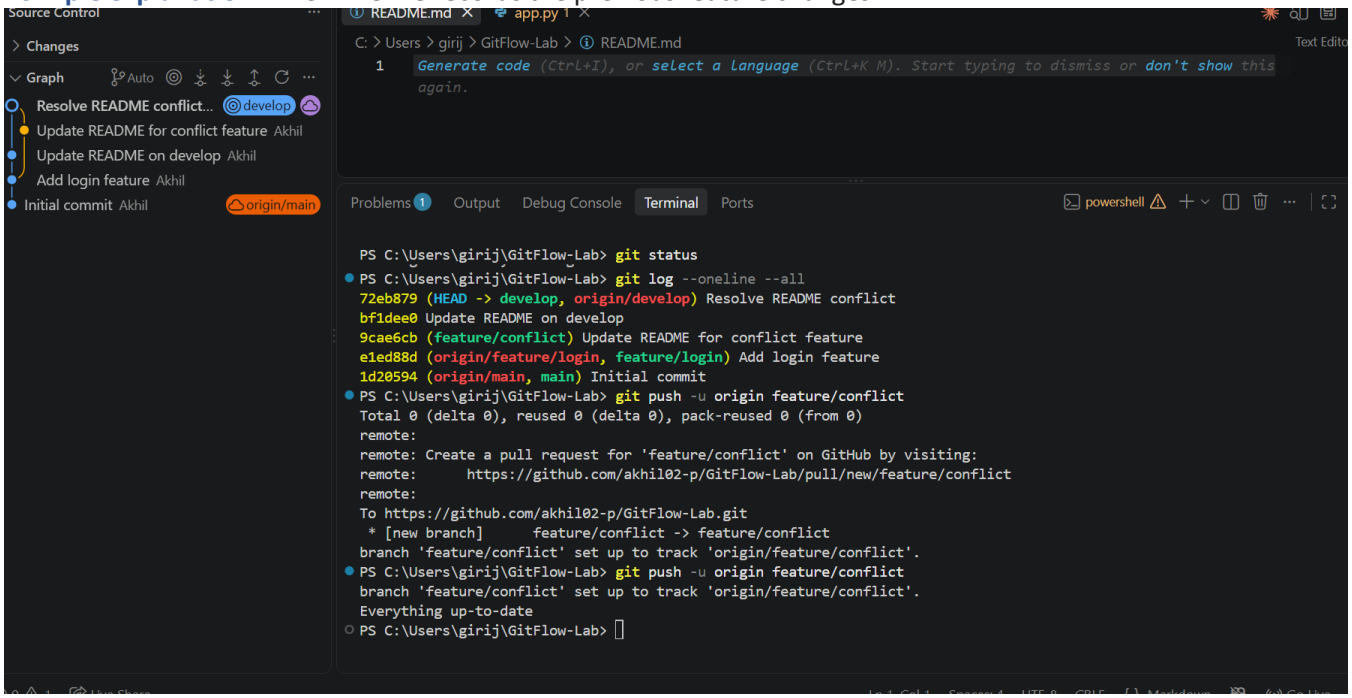


Screenshot 48 — Step 48

Step 49. Review the Conflict-Feature History

What you see: The VS Code Timeline shows the conflict-feature README update and saved files.

Simple explanation: The Timeline records the previous feature changes.

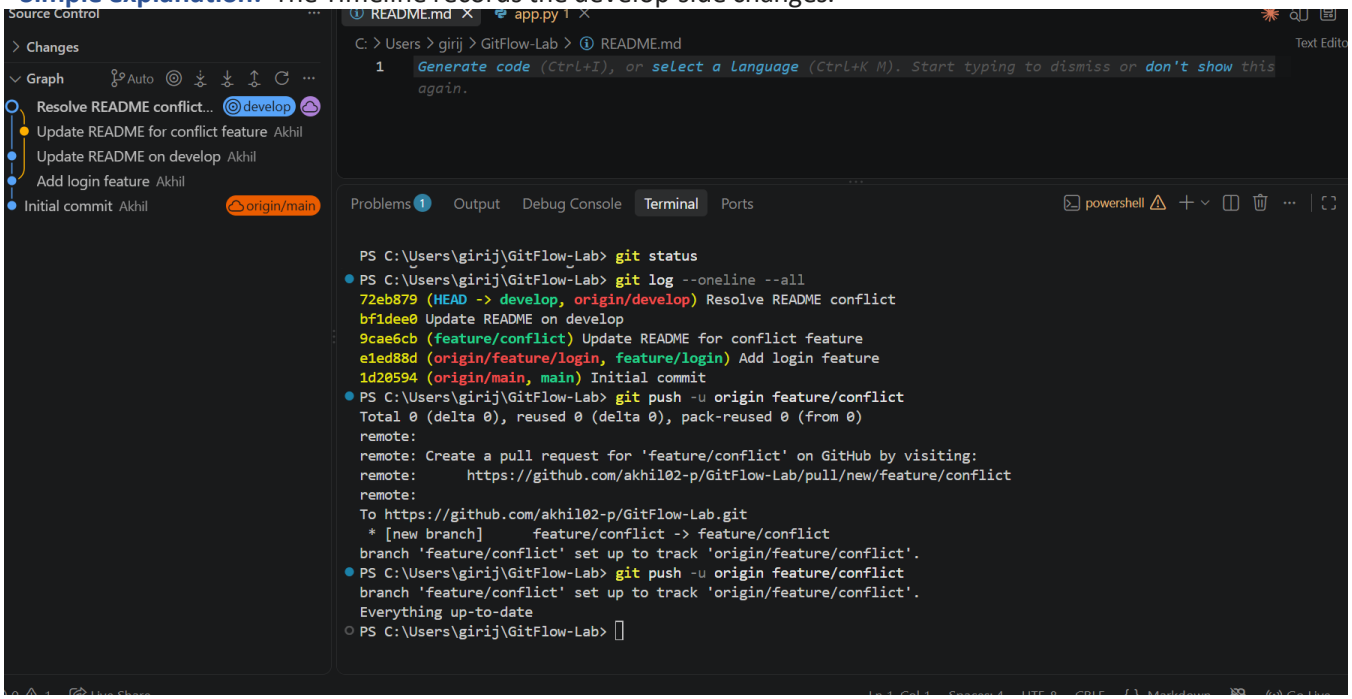


Screenshot 49 — Step 49

Step 50. Review the Develop History

What you see: The VS Code Timeline shows the develop README update and saved files.

Simple explanation: The Timeline records the develop-side changes.

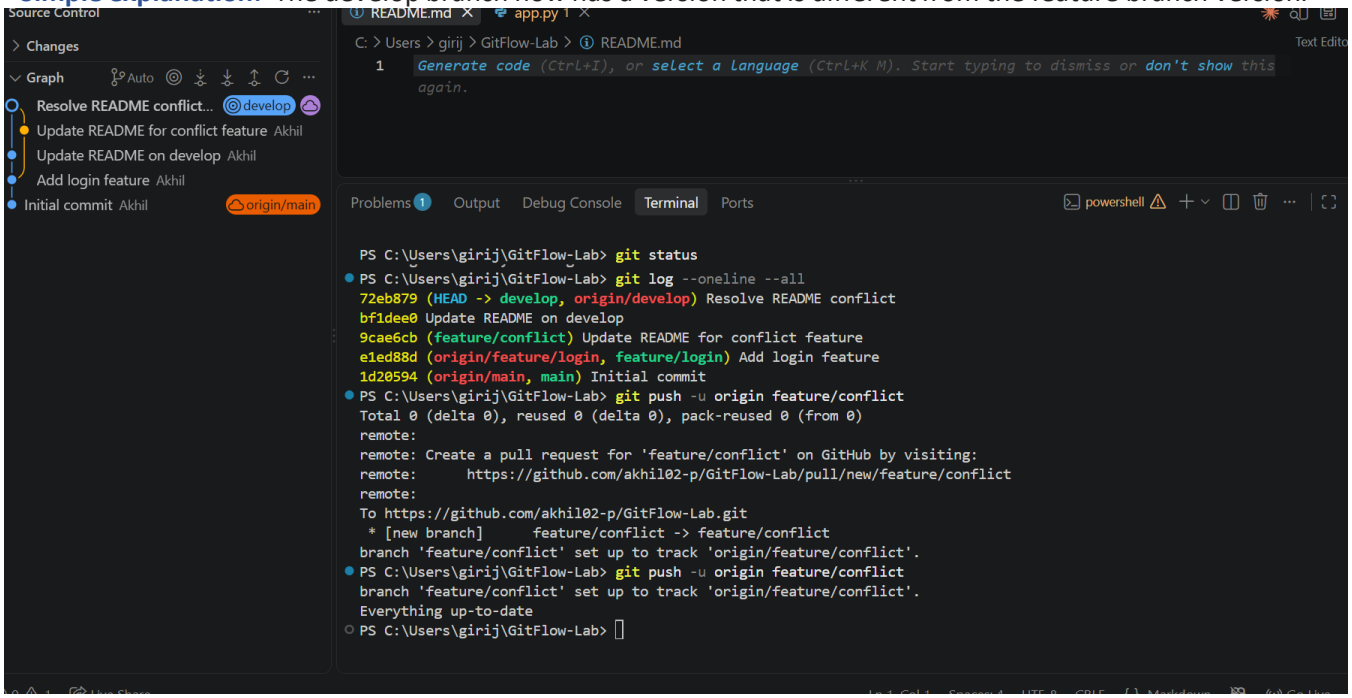


Screenshot 50 — Step 50

Step 51. Create Another Develop Version

What you see: README.md shows "Gitflow Lab 5 - Another Develop Version".

Simple explanation: The develop branch now has a version that is different from the feature branch version.



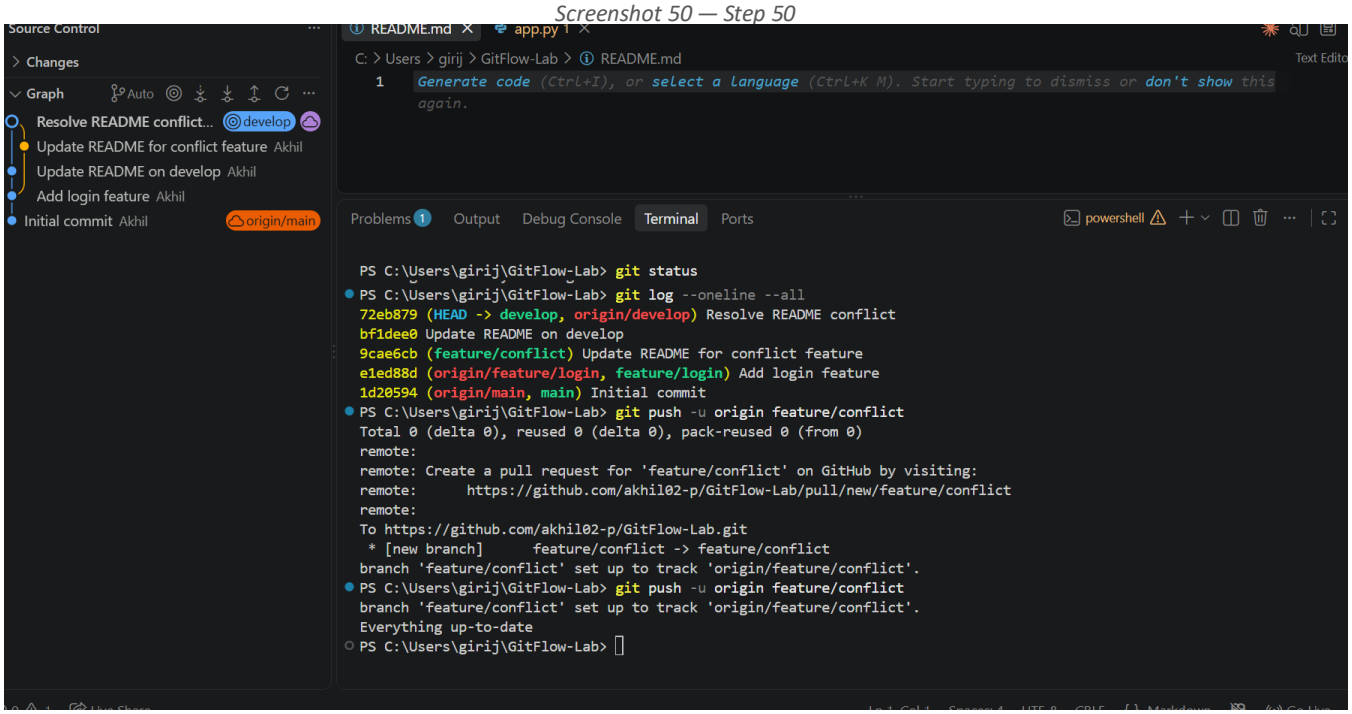
Screenshot 51 — Step 51

Step 52. Open the Merge Editor

What you see: VS Code Merge Editor shows Incoming develop content and Current feature/conflict content for README.md.

Simple explanation: Git cannot automatically decide which version to keep. The Merge Editor shows both sides.

Screenshot 50 — Step 50

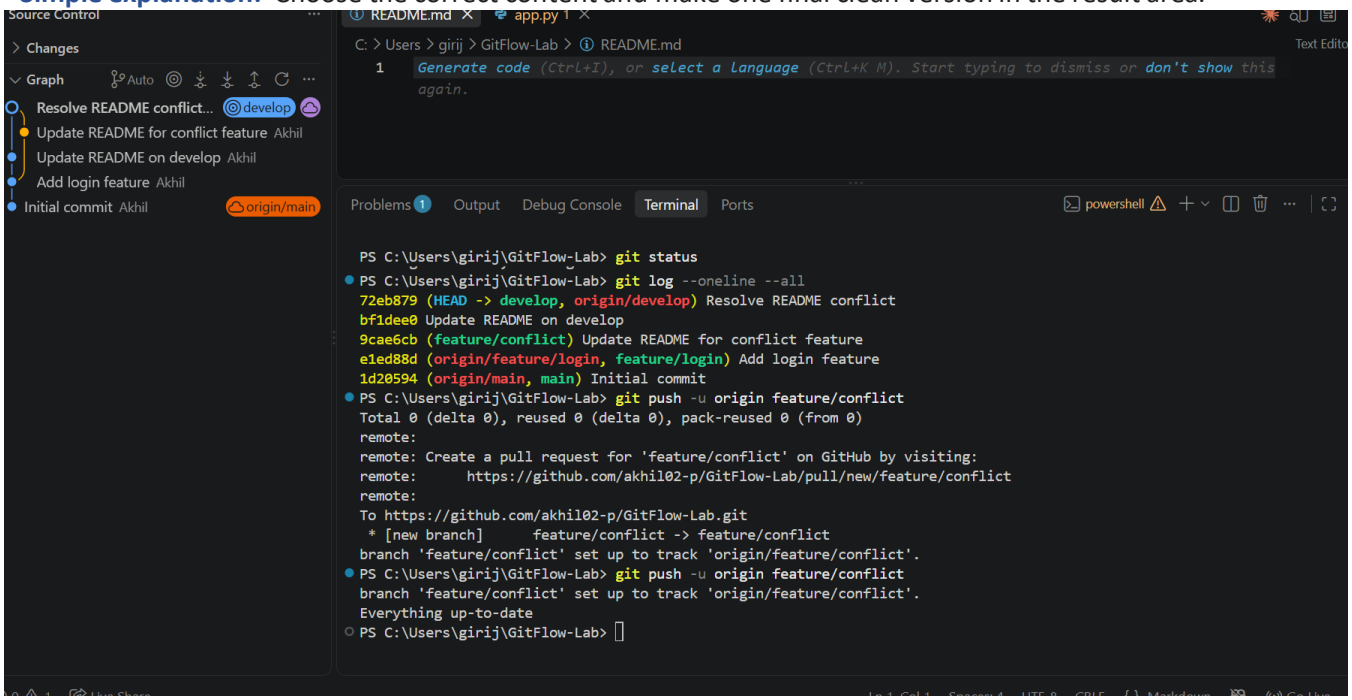


Screenshot 52 — Step 52

Step 53. See Both Versions Together

What you see: The Merge Editor shows the two README versions and the result area.

Simple explanation: Choose the correct content and make one final clean version in the result area.



Screenshot 53 — Step 53

Step 54. Complete the Conflict Resolution in Source Control

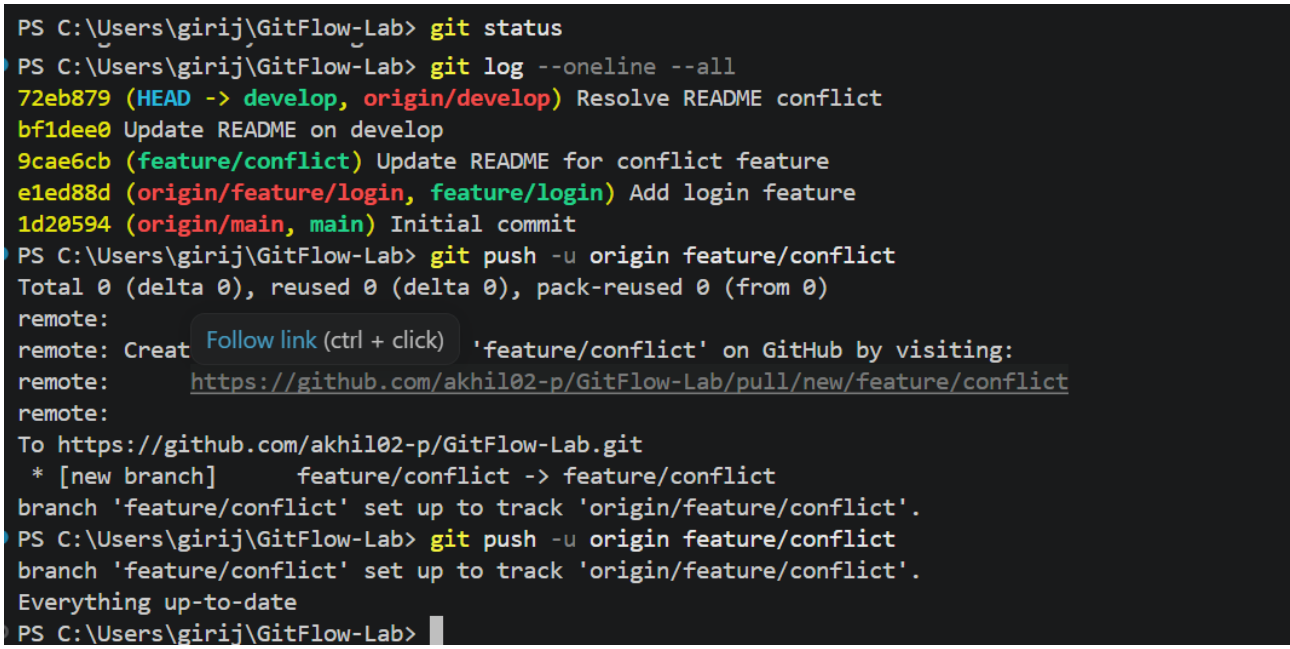
What you see: VS Code Source Control shows the merge state and the staged README.md.

Simple explanation: After the file is corrected, it must be staged and the merge must be completed with a commit.

Command:

```
git add README.md
```

Purpose: Stages the resolved README file.

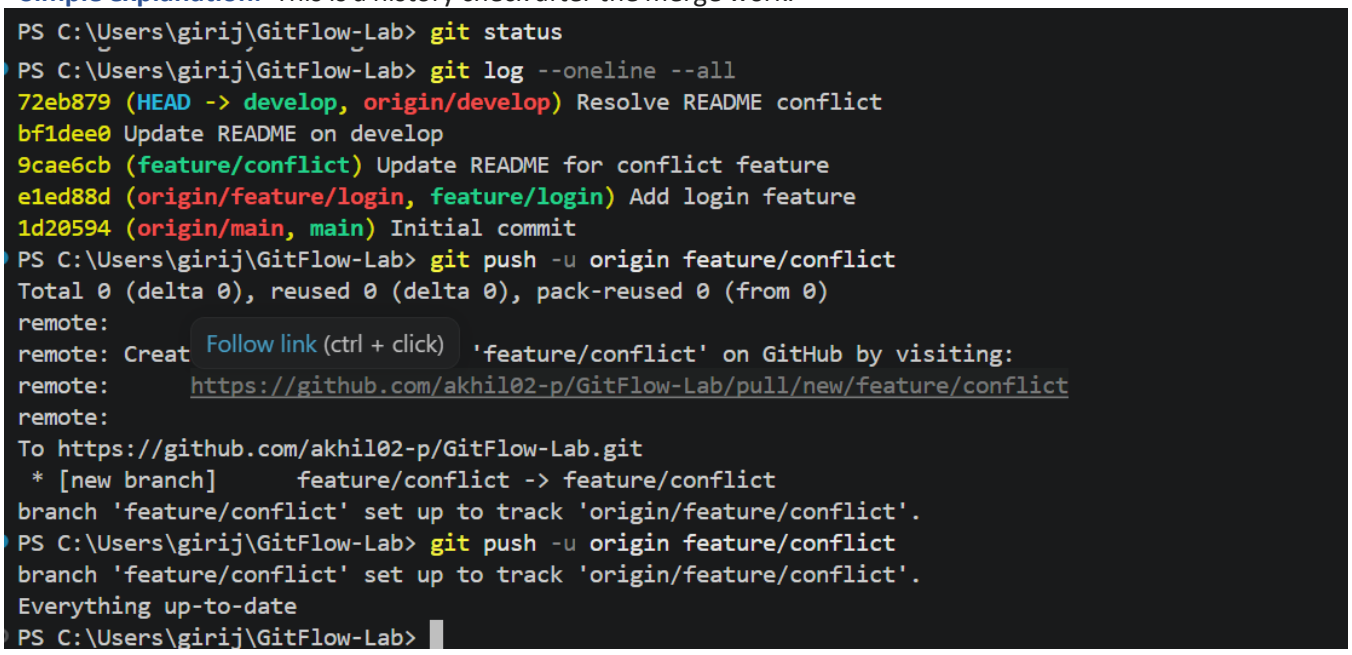


Screenshot 54 — Step 54

Step 55. Review the Merge History

What you see: The terminal and Source Control show the merge history and Fast-forward/app.py information.

Simple explanation: This is a history check after the merge work.



Screenshot 55 — Step 55

Step 56. Review the Final Local History

What you see: VS Code shows the project history after the conflict workflow.

Simple explanation: The history should now contain the work from the branches and the merge.

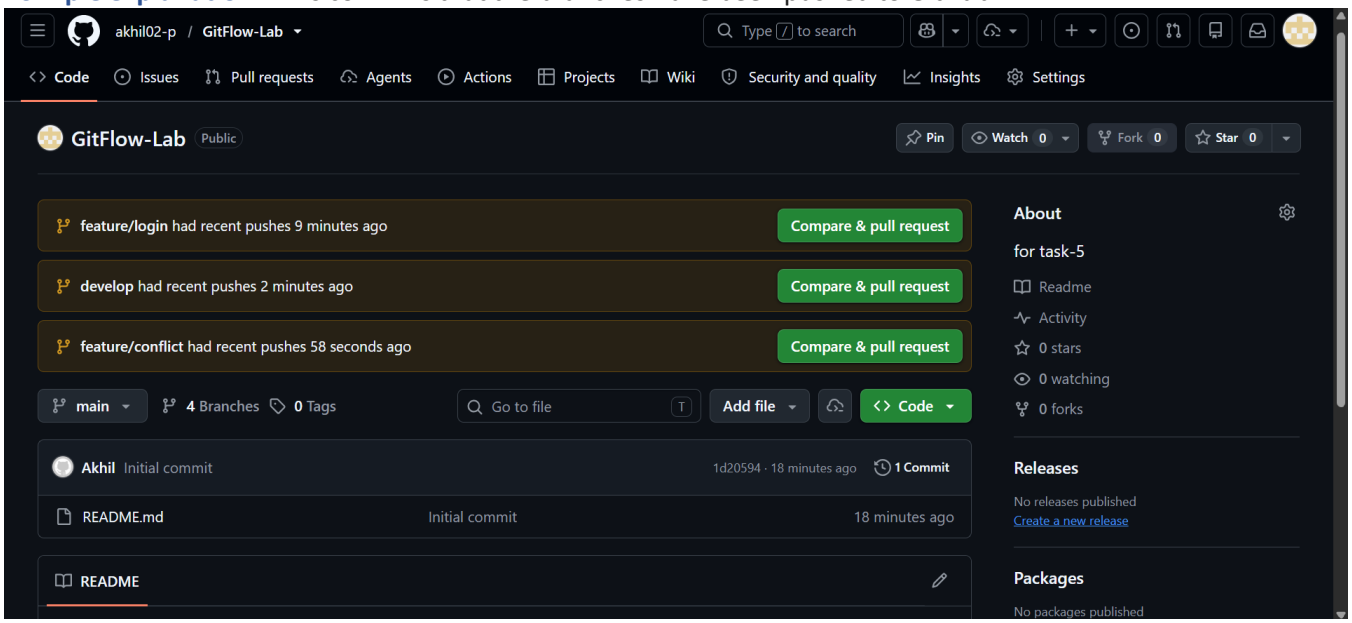
```
PS C:\Users\girij\GitFlow-Lab> git status
PS C:\Users\girij\GitFlow-Lab> git log --oneline --all
72eb879 (HEAD -> develop, origin/develop) Resolve README conflict
bf1dee0 Update README on develop
9cae6cb (feature/conflict) Update README for conflict feature
e1ed88d (origin/feature/login, feature/login) Add login feature
1d20594 (origin/main, main) Initial commit
PS C:\Users\girij\GitFlow-Lab> git push -u origin feature/conflict
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Creat  Follow link (ctrl + click)  'feature/conflict' on GitHub by visiting:
remote:      https://github.com/akhil02-p/GitFlow-Lab/pull/new/feature/conflict
remote:
To https://github.com/akhil02-p/GitFlow-Lab.git
 * [new branch]      feature/conflict -> feature/conflict
branch 'feature/conflict' set up to track 'origin/feature/conflict'.
PS C:\Users\girij\GitFlow-Lab> git push -u origin feature/conflict
branch 'feature/conflict' set up to track 'origin/feature/conflict'.
Everything up-to-date
PS C:\Users\girij\GitFlow-Lab>
```

Screenshot 56 — Step 56

Step 57. Check GitHub Branches and Recent Pushes

What you see: GitHub shows recent pushes for feature/conflict and develop.

Simple explanation: This confirms that the branches have been pushed to GitHub.

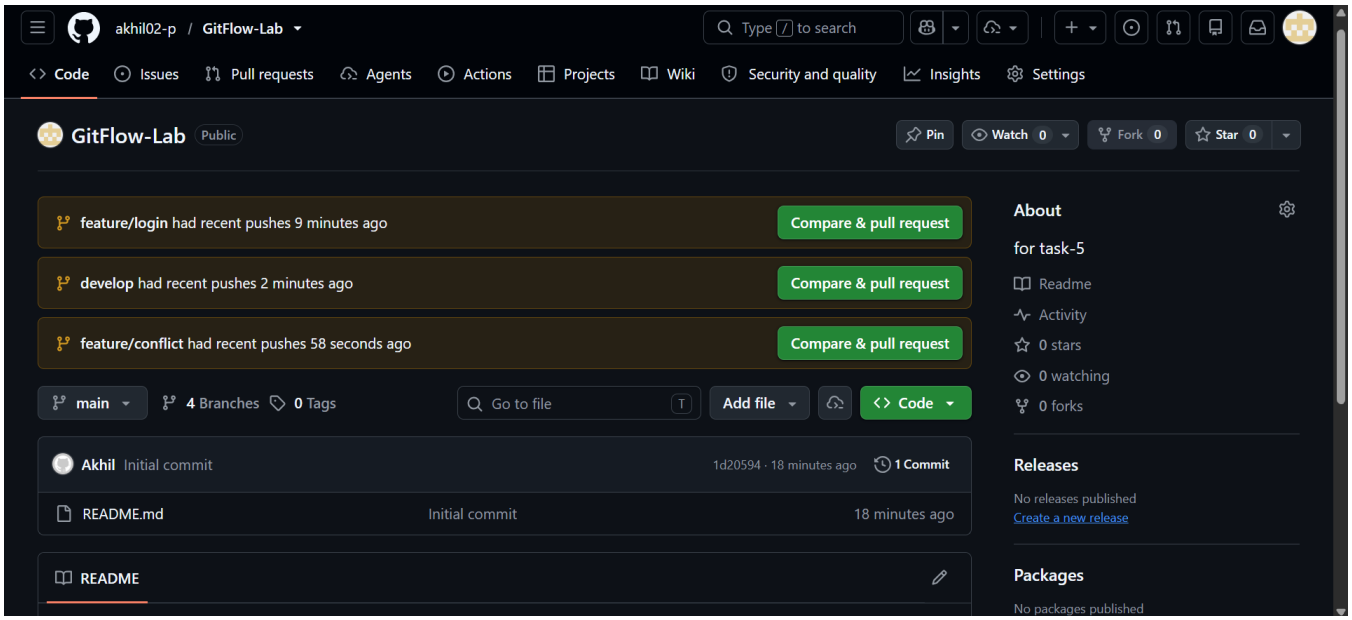


Screenshot 57 — Step 57

Step 58. Open the Final develop Branch

What you see: GitHub shows the develop branch with README.md, app.py, the commit history, and the README content.

Simple explanation: This is the final verification screen. The develop branch contains the integrated project work.



Screenshot 58 — Step 58

Final Revision

Remember the main flow: Create GitHub repository → connect the local project → create develop → create feature/login → commit → push → merge into develop → create feature/conflict → make different changes → merge → resolve the conflict → stage → commit → push → verify on GitHub.

Most Important Commands

- `git status` —
- `git branch` —
- `git checkout -b branch-name` —
- `git add file-name` —
- `git commit -m "message"` —
- `git push` —
- `git merge branch-name` —
- `python app.py` —